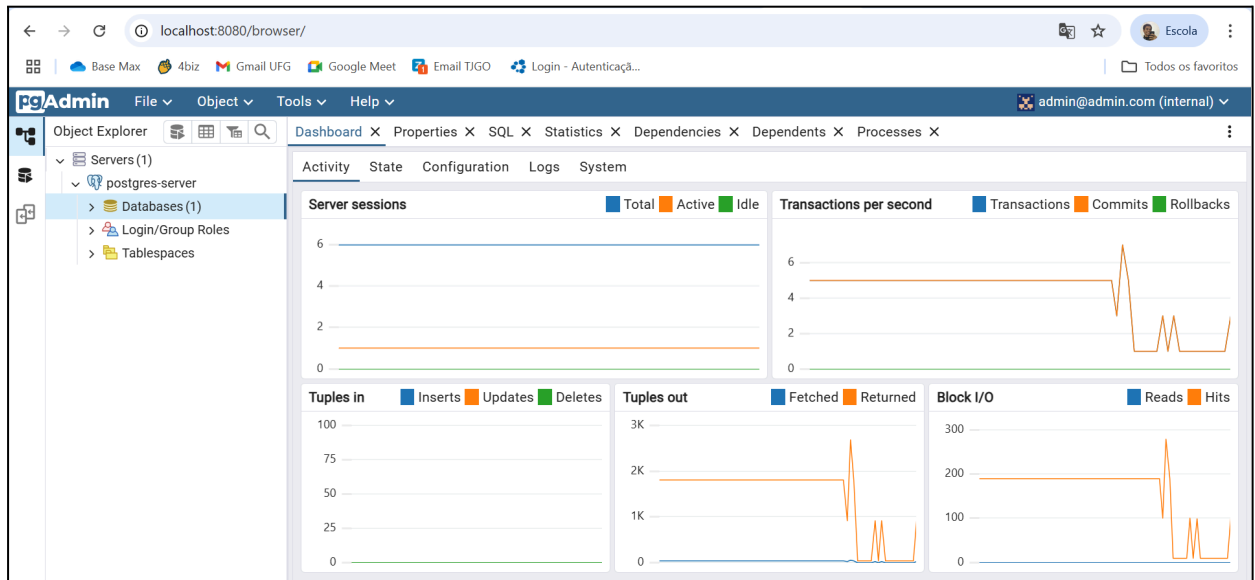


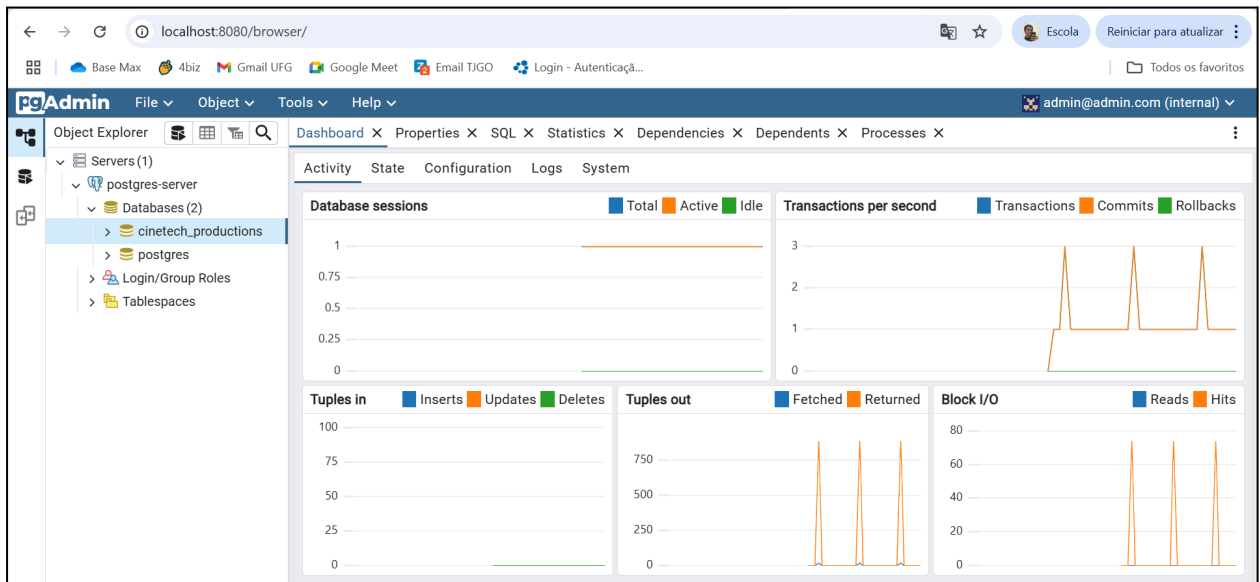
Aluno: Kevin Rodrigues Ribeiro
Matrícula: 2025200242

Fase 1: Projeto e Implementação do Banco de Dados

● Subindo PgAdmin e Postgresql;



● Passo 1: Criar o Banco de Dados Principal



• Passo 2: Projetar o Esquema de Dados Brutos

Criando Esquema

The screenshot shows the pgAdmin 4 web interface. In the left-hand 'Object Explorer', the 'Schemas (2)' folder is expanded, showing 'public' and 'raw_data'. The 'raw_data' schema is selected. The central pane displays the 'Query' editor with the following SQL command:

```
1 CREATE SCHEMA raw_data;
```

The 'Data Output' pane at the bottom shows the execution result: 'CREATE SCHEMA' and 'Query returned successfully in 64 msec.' The status bar at the bottom indicates 'Total rows: Query complete 00:00:00.064'.

Criando tabelas Produção, Pessoas e Equipe.

The screenshot shows the pgAdmin 4 web interface. In the left-hand 'Object Explorer', the 'raw_data' schema is expanded, showing 'equipe', 'pessoa', and 'producao' tables. The 'equipe' table is selected. The central pane displays the 'Query' editor with the following SQL command:

```
1 CREATE TABLE raw_data.equipe (  
2     pessoaID INT,  
3     producaoID INT,  
4     papel TEXT,  
5     PRIMARY KEY (pessoaID, producaoID),  
6     FOREIGN KEY (pessoaID) REFERENCES raw_data.pessoa(pessoaID),  
7     FOREIGN KEY (producaoID) REFERENCES raw_data.producao(producaoID)  
8 );
```

The 'Data Output' pane at the bottom shows the execution result: 'CREATE TABLE' and 'Query returned successfully in 101 msec.' The status bar at the bottom indicates 'Total rows: Query complete 00:00:00.101'.

● Passo 3: Ingestão de Dados

Ingestão de dados “Equipe”

The screenshot shows the PgAdmin interface with the 'equipe' table selected in the Object Explorer. The table is located in the 'cinetech_productions/postgres@postgres-server' database. The table structure is as follows:

	pessoaid	producaoid	papel
	[PK] integer	[PK] integer	text
1	715190	37497	Nadine's Client
2	715190	124692	[null]
3	715190	258179	Reporter
4	715190	516564	The Butler
5	715190	683371	Palace Guard
6	715190	874192	Man with Umbrella
7	715191	22384	Man

Ingestão de dados “Pessoa”

The screenshot shows the PgAdmin interface with the 'pessoa' table selected in the Object Explorer. The table is located in the 'cinetech_productions/postgres@postgres-server' database. The table structure is as follows:

	pessoaid	nome
	[PK] integer	character varying (255)
1	2523	Accardi, A.J.
2	3205	Acuna, Ramón
3	2725	Achard, Olivier
4	156	A., David
5	360	Aaron, James
6	2695	Aceves, Gustavo
7	2970	Ackman, Felix
8	174	A.J.

Ingestão de dados “producao”

The screenshot shows the PgAdmin interface with the 'producao' table selected in the Object Explorer. The table is located in the 'cinetech_productions/postgres@postgres-server' database. The table structure is as follows:

	producaoid	titulo	ano_producao	tip
	[PK] integer	character varying (255)	integer	int
1	70625	Campanile d'oro, Il		1955
2	76172	Calda libre		1994
3	97851	Crucea de piatra		1993
4	86809	Clinic, The		2007
5	79563	Chauffeur, The		1921
6	84114	Chyorny kvadrat		1988
7	2718	20 Minuten Aufenthalt		1960
8	76511	Celestial Desires		2003

Fase 2: Projeto do Esquema Analítico

- **Passo 4: Criar o Esquema de Analytic**

The screenshot shows the pgAdmin interface with the 'cinetech_productions/postgres@postgres-server' connection selected. The 'Query' tab is active, displaying the following SQL code:

```
1 -- Excluir a tabela 'animations' se ela já existir
2 DROP TABLE IF EXISTS analytics.animations;
3
4 -- Criar a tabela 'animations'
5 CREATE TABLE analytics.animations (
6     producaoID INT PRIMARY KEY,
7     titulo VARCHAR(255),
8 );
```

The 'Messages' tab shows the following output:

```
NOTICE: table "animations" does not exist, skipping
INSERT 0 0
Query returned successfully in 715 msec.
```

Fase 3: Gerenciamento de Usuários e Controle de Acesso

- **Passo 5: Criar Usuários Especializados**

The screenshot shows the pgAdmin interface with the 'cinetech_productions/postgres@postgres-server' connection selected. The 'Login/Group Roles' list is expanded, showing 22 roles. The 'Data Output' tab is active, displaying a table of granted privileges:

grantee name	table_schema	table_name	privilege_type
postgres	analytics	short_films	REFERENCES
postgres	analytics	short_films	TRUNCATE
postgres	analytics	short_films	DELETE
postgres	analytics	short_films	SELECT
postgres	analytics	short_films	UPDATE
postgres	analytics	short_films	INSERT
postgres	analytics	theater_productions	DELETE
postgres	analytics	theater_productions	TRUNCATE
postgres	analytics	theater_productions	REFERENCES
postgres	analytics	theater_productions	TRIGGER
postgres	analytics	theater_productions	INSERT

Fase 4: Analytics e Relatórios

● Passo 6: Criar Views Analíticas

The screenshot shows the PgAdmin interface with the 'Object Explorer' on the left. The 'Views (4)' folder is expanded, showing 'crew_analysis', 'production_summary', 'top_actors_by_type', and 'yearly_production_trends'. The 'Query' tab is active, displaying a SQL query:

```
6 FROM
7   raw_data.equipe
8 WHERE
9   papel IS NOT NULL AND papel != 'null'
10 GROUP BY
11   papel
12 ORDER BY
13   total_pessoas DESC;
```

The 'Data Output' tab shows the message: 'Query returned successfully in 107 msec.' The status bar at the bottom indicates 'Total rows: Query complete 00:00:00.107'.

● Passo 7: Realizar Análise de Dados

1. Qual tipo de produção tem mais produções?

The screenshot shows the PgAdmin interface with the 'Query' tab active. The SQL query is:

```
1 SELECT tipo_ID, total_producoes
2 FROM analytics.production_summary
3 ORDER BY total_producoes DESC
4 LIMIT 1;
```

The 'Data Output' tab shows the result of the query:

tipo_id	total_producoes
7	467482

The status bar at the bottom indicates 'Showing rows: 1 to 1' and 'Page No: 1 of 1'.

2. Quais são os 10 atores mais ativos em todos os tipos de produção?

The screenshot shows the pgAdmin interface with a SQL query executed. The query is as follows:

```
3 SUM(total_producoes) AS total_participacoes
4 FROM
5 analytics.top_actors_by_type
6 GROUP BY
7 nome_pessoa
8 ORDER BY
9 total_participacoes DESC
10 LIMIT 10;
```

The data output table shows the following results:

nome_pessoa	total_participacoes
Vorderman, Carol	2428
Stern, Howard	2141
Carson, Johnny	2001
Quigley, Robin	1801

3. Como o número de produções mudou nos últimos 50 anos?

The screenshot shows the pgAdmin interface with a SQL query executed. The query is as follows:

```
2 ano_producao,
3 total_producoes
4 FROM
5 analytics.yearly_production_trends
6 WHERE
7 ano_producao >= EXTRACT(YEAR FROM CURRENT_DATE) - 50
8 ORDER BY
9 ano_producao ASC;
```

The data output table shows the following results:

ano_producao	total_producoes
1975	7827
1976	7362
1977	7329
1978	7489

4. Quais anos tiveram a maior produção?

The screenshot shows the PgAdmin interface with a SQL query executed. The query is:

```
SELECT
ano_producao,
total_producoes
FROM
analytics.yearly_production_trends
ORDER BY
total_producoes DESC;
```

The results are displayed in a table with 2 columns: ano_producao (integer) and total_producoes (bigint). The top 4 rows are shown:

ano_producao	total_producoes
2006	35150
2002	31180
2001	28742
2000	26184

Total rows: 128. Query complete 00:00:00.146.

5. Qual porcentagem de produções tem membros da equipe com papéis específicos?

The screenshot shows the PgAdmin interface with a SQL query executed. The query is:

```
raw_data.equipe AS e
WHERE
e.papel IS NOT NULL
GROUP BY
e.papel
ORDER BY
porcentagem_producoes DESC;
```

The results are displayed in a table with 2 columns: papel (text) and porcentagem_producoes (numeric). The top 3 rows are shown:

papel	porcentagem_producoes
Himself	12.585872045767
Herself	8.716647649118
Himself - Host	3.304559522320

Total rows: 1868943. Query complete 00:00:20.925.

Fase 5: Performance e Otimização (Extra, envolve pesquisa)

- **Passo 8: Otimização de Performance**

The screenshot shows the pgAdmin interface with a SQL query being executed. The query is:

```
4  p.nome,  
5  e.papel  
6  FROM  
7  raw_data.equipe AS e  
8  JOIN  
9  raw_data.pessoa AS p ON e.pessoaID = p.pessoaID  
10 WHERE  
11 e.papel = 'Neo';
```

The query plan shows the following steps:

Step	Operation	Cost	Rows	Width	Actual Time	Loops
1	Gather	(cost=1000.43..130678.22 rows=163 width=31)	163	31	49.447..1030.469	19
2	Workers Planned: 2					
3	Workers Launched: 2					
4	Nested Loop	(cost=0.43..129661.92 rows=68 width=31)	68	31	58.849..990.126	6

Total rows: 15. Query complete 00:00:01.187.

- **Passo 9: Estratégia de Backup e Recuperação**

The screenshot shows the pgAdmin interface with the 'Backup' dialog box open. The dialog box has a 'Select file' section with a table showing the backup file:

Name	Date Modified	Size
Backup	Tue Aug 12 00:25:14 2025	

The 'Save As' field is empty. The 'File Format' is set to 'All Files'. The 'Create' button is highlighted.

Fase 6: Documentação e Submissão

O esquema do banco é dividido em dois: **raw_data** para os dados brutos e **analytics** para as tabelas analíticas.

Esquema raw_data

- **produção:** Contém informações de produção, com chave primária.
- **pessoa:** Contém informações de pessoas, com chave primária pessoalID.
- **equipe:** Contém informações de equipe/elenco, com chave primária (pessoalID, producaoID) e chaves estrangeiras que referenciam producao e pessoa.

Esquema analytics

- Tabelas: As 9 tabelas (movies, tv_shows, etc.) especializadas para cada tipo de produção, contendo campos relevantes da tabela producao.

Estratégias de Indexação

Índices foram criados automaticamente nas chaves primárias. Para otimização de performance, índices adicionais devem ser considerados em colunas usadas em cláusulas WHERE ou ORDER BY.

Documentação de Acesso de Usuários / Scripts de Criação de Usuários

SQL

```
CREATE ROLE analyst_movies WITH LOGIN PASSWORD 'analyst_movies_pass';
CREATE ROLE analyst_tv WITH LOGIN PASSWORD 'analyst_tv_pass';
CREATE ROLE analyst_games WITH LOGIN PASSWORD 'analyst_games_pass';
CREATE ROLE analyst_docs WITH LOGIN PASSWORD 'analyst_docs_pass';
CREATE ROLE analyst_all WITH LOGIN PASSWORD 'analyst_all_pass';
CREATE ROLE data_scientist WITH LOGIN PASSWORD 'data_scientist_pass';
```

Matriz de Permissões

Usuário	Permissão	Tabelas Acessadas
analyst_movies	SELECT	analytics.movies
analyst_tv	SELECT	analytics.tv_shows
analyst_all	SELECT	Todas as tabelas em analytics
data_scientist	ALL PRIVILEGES	Todas as tabelas em analytics

Documentação de Ingestão de Dados / Scripts de Ingestão

```
import psycopg2
import os
from psycopg2 import sql
import io
import re
import tempfile

# Dados de conexão com o banco de dados
```

```

DB_HOST = os.getenv("DB_HOST", "postgresql")
DB_NAME = os.getenv("DB_NAME", "cinetech_productions")
DB_USER = os.getenv("DB_USER", "postgres")
DB_PASS = os.getenv("POSTGRES_PASSWORD", os.getenv("DB_PASSWORD",
"postgres123"))
DB_SCHEMA = os.getenv("DB_SCHEMA", "raw_data")

# Caminho dos arquivos de dados dentro do contêiner
data_path = "/app"

# Conecta ao banco de dados
try:
    conn = psycopg2.connect(
        host=DB_HOST,
        database=DB_NAME,
        user=DB_USER,
        password=DB_PASS,
        client_encoding='LATIN1'
    )
    cur = conn.cursor()
    print("Conexão com o banco de dados estabelecida com sucesso.")
    # Diagnóstico do ambiente de conexão
    cur.execute("select current_database(), current_user,
inet_server_addr(), version()")
    db, usr, host_addr, ver = cur.fetchone()
    print(f"Diagnostico: database={db}, user={usr},
host_addr={host_addr}")
    print(ver.split('\n')[0])
except (Exception, psycopg2.Error) as error:
    print("Erro ao conectar ao PostgreSQL:", error)
    exit()

def quote_ident_if_needed(identifier: str) -> str:
    """
    Retorna o identificador com aspas duplas se contiver caracteres
    especiais
    (ex.: espaço) ou letras maiúsculas. Caso contrário, retorna sem
    aspas.
    """
    # Aspas dentro do nome (raro) precisam ser duplicadas para serem
    válidas em SQL
    sanitized = identifier.replace("'", '""')

```

```

    needs_quotes = (sanitized != sanitized.lower()) or any(ch not in
"abcdefghijklmnopqrstuvwxyz0123456789_" for ch in sanitized)
    return f'"{sanitized}"' if needs_quotes else sanitized

def make_table_name(schema: str, table: str) -> str:
    return
f'"{quote_ident_if_needed(schema)}.{quote_ident_if_needed(table)}"'

def ingest_data_copy_from(file_name, table_name, columns):
    file_path = os.path.join(data_path, file_name)
    print(f"Iniciando a ingestao de dados de {file_name} para a tabela
{table_name}...")
    # Confirma se a tabela existe antes do COPY
    try:
        cur.execute("select to_regclass(%s)", (table_name,))
        reg = cur.fetchone()[0]
        print(f"Checagem existencia: to_regclass('{table_name}') =
{reg}")
    except Exception as e:
        print(f"Falha ao checar existencia de {table_name}: {e}")

    # Monta COPY usando composicao segura de identificadores (schema,
tabela, colunas)
    def _split_schema_table(full_name: str):
        # Suporta formas como raw_data.producao ou "raw
data"."Producao"
        if '.' not in full_name:
            return None, full_name
        schema_part, table_part = full_name.split('.', 1)
        return schema_part.strip('"'), table_part.strip('"')

    schema_part, table_part = _split_schema_table(table_name)

    # Resolve nomes reais das colunas no catálogo, de forma
case-insensitive
    def _resolve_columns(real_schema: str, real_table: str,
desired_cols):
        try:
            qname =
f'"{quote_ident_if_needed(real_schema)}.{quote_ident_if_needed(real_tabl
e)}"'

            cur.execute("""
                select a.attname

```

```

        from pg_attribute a
        join pg_class c on c.oid = a.attrelid
        join pg_namespace n on n.oid = c.relnamespace
        where n.nspname = %s
        and c.relname = %s
        and a.attnum > 0 and not a.attisdropped
        """, (real_schema, real_table))
    existing = [r[0] for r in cur.fetchall()]
    mapping = {e.lower(): e for e in existing}
    resolved = []
    for d in desired_cols:
        key = d.lower()
        if key in mapping:
            resolved.append(mapping[key])
        else:
            # fallback: usar minúsculo do desejado
            resolved.append(key)
    if [c for c in resolved] != desired_cols:
        print(f"Colunas ajustadas para tabela {qname}: {resolved}")

    return resolved
except Exception as e:
    print(f"Falha ao resolver colunas para {real_schema}.{real_table}: {e}")
    return [c.lower() for c in desired_cols]

resolved_columns = _resolve_columns(schema_part or DB_SCHEMA,
table_part, columns)

columns_ident_list = [sql.Identifier(c) for c in resolved_columns]
columns_sql = sql.SQL(', ').join(columns_ident_list)
# Estratégia robusta: COPY para tabela temporária (tudo TEXT),
depois INSERT com CAST e ON CONFLICT DO NOTHING
tmp_table = f"tmp_{table_part}_{os.getpid()}"
tmp_ident = sql.Identifier(tmp_table)
target_schema_ident = sql.Identifier(schema_part) if schema_part
else sql.Identifier(DB_SCHEMA)
target_table_ident = sql.Identifier(table_part)

# 1) Criar tabela temporária com colunas TEXT
create_cols_sql = sql.SQL(', ').join([sql.SQL("{}
TEXT").format(sql.Identifier(c)) for c in resolved_columns])

```

```

        create_tmp_stmt = sql.SQL("CREATE TEMP TABLE {} ({})) ON COMMIT
DROP").format(tmp_ident, create_cols_sql)

    # 2) COPY para a tabela temporária com delimitador TAB e NULL
    'null'

    copy_tmp_stmt = sql.SQL("COPY {} ({})) FROM STDIN WITH (FORMAT text,
DELIMITER E'\\t', NULL 'null')").format(
        tmp_ident,
        columns_sql
    )

    # 3) Descobrir tipos das colunas alvo para aplicar CASTs
    cur.execute(
        """
        select column_name, data_type
        from information_schema.columns
        where table_schema = %s and table_name = %s
        """,
        (schema_part or DB_SCHEMA, table_part)
    )
    type_map = {r[0]: r[1] for r in cur.fetchall()}

    # 4) Montar SELECT com casts apropriados
    select_exprs = []
    for c in resolved_columns:
        dtype = (type_map.get(c) or '').lower()
        id_c = sql.Identifier(c)
        if any(t in dtype for t in ["integer", "smallint", "bigint"]):
            # remove tudo que não for dígito ou sinal e faz cast
            expr = sql.SQL("NULLIF(regex_replace({col}, '[^0-9-]', ''),
'g'), '')::integer").format(col=id_c)
        elif any(t in dtype for t in ["double", "real", "numeric",
"decimal"]):
            expr = sql.SQL("NULLIF(regex_replace({col},
'[^0-9eE+\\-\\.]', ''), 'g'), '')::numeric").format(col=id_c)
        else:
            expr = id_c
        select_exprs.append(expr)
    select_list_sql = sql.SQL(', ').join(select_exprs)

    insert_stmt = sql.SQL("""
        INSERT INTO {}.{} ({}))
        SELECT {} FROM {}
    """)

```

```

        ON CONFLICT DO NOTHING
    """).format(
        target_schema_ident,
        target_table_ident,
        columns_sql,
        select_list_sql,
        tmp_ident
    )

    try:
        # Cria temp table
        cur.execute(create_tmp_stmt)

        # 0) Contar total de linhas para calcular percentual
        total_linhas = 0
        with open(file_path, 'r', encoding='latin1') as fcnt:
            for _ in fcnt:
                total_linhas += 1
        if total_linhas == 0:
            print(f"{file_name}: arquivo vazio. Nada a carregar.")
            return

        # 1) Pré-processar para arquivo temporário com progressão
percentual
        expected_cols = len(resolved_columns)
        tmp_fd, tmp_path =
tempfile.mkstemp(prefix=f"ing_{table_part}_", suffix=".tsv")
        os.close(tmp_fd) # vamos reabrir como texto
        processed = 0
        next_mark = 0 # próximo percentual a imprimir
        with open(file_path, 'r', encoding='latin1') as fin,
open(tmp_path, 'w', encoding='latin1', newline='') as fout:
            for raw_line in fin:
                # transformar delimitadores '##' (2+ '#') em TAB
                line = re.sub(r"#{2,}", "\t", raw_line.rstrip("\n\r"))
                parts = line.split('\t') if line != '' else ['']
                if len(parts) > expected_cols:
                    # Unir o excedente com ESPAÇO (não usar TAB, para
não criar delimitadores extras)
                    parts = parts[:expected_cols-1] + ['
'.join(parts[expected_cols-1:])]
                elif len(parts) < expected_cols:
                    parts = parts + [''] * (expected_cols - len(parts))

```

```

        fout.write('\t'.join(parts) + '\n')
        processed += 1
        # imprimir a cada 5%
        if total_linhas:
            pct = int(processed * 100 / total_linhas)
            if pct >= next_mark:
                print(f"{file_name}: preparando dados {pct}%
({processed}/{total_linhas})")
                next_mark += 5

    # 2) COPY para temp
    print(f"{file_name}: copiando para tabela temporaria...")
    with open(tmp_path, 'r', encoding='latin1') as buf:
        cur.copy_expert(copy_tmp_stmt, buf)
    print(f"{file_name}: COPY concluido.")

    # 3) Inserir no destino com casts e ignorando duplicatas
    print(f"{file_name}: inserindo na tabela de destino...")
    cur.execute(insert_stmt)
    inseridas = cur.rowcount if cur.rowcount is not None else 0
    print(f"{file_name}: inseridas {inseridas} linhas (de
{total_linhas}).")

    conn.commit()
    print(f"Ingestao de dados de {file_name} concluida com
sucesso.")
    try:
        os.remove(tmp_path)
    except Exception:
        pass
except FileNotFoundError:
    print(f"Erro: O arquivo {file_path} nao foi encontrado.")
except Exception as e:
    conn.rollback()
    print(f"Erro durante a ingestao de {file_name}: {e}")

# Definição das tabelas e arquivos para ingestão
ingest_config = [
    {
        'file': 'producao.txt',
        'table': make_table_name(DB_SCHEMA, 'producao'),
        'columns': ['producaoID', 'titulo', 'ano_producao', 'tipo_ID']
    },

```

```

{
    'file': 'pessoa.txt',
    'table': make_table_name(DB_SCHEMA, 'pessoa'),
    'columns': ['pessoaID', 'nome']
},
{
    'file': 'equipe.txt',
    'table': make_table_name(DB_SCHEMA, 'equipe'),
    'columns': ['pessoaID', 'producaoID', 'papel']
}
]

# Executa a ingestão para cada arquivo
for config in ingest_config:
    ingest_data_copy_from(config['file'], config['table'],
config['columns'])

# Fecha a conexão com o banco de dados
cur.close()
conn.close()

```

Verificações de Qualidade dos Dados

O script lida com o delimitador ## e trata valores null corretamente, inserindo-os como NULL no banco de dados.

Procedimentos de Tratamento de Erros

O script possui blocos `try...except` para lidar com erros de conexão e falhas na ingestão, garantindo que o `conn.rollback()` seja executado para reverter transações em caso de erro.

Documentação de Desempenho / Estratégias de Otimização

A análise de desempenho deve ser realizada com `EXPLAIN ANALYZE` para identificar gargalos em consultas complexas.

